

C++

A Linguagem que Moldou o Desenvolvimento
de Software Moderno

SEMINÁRIO ACADÊMICO

Apresentado por:

Gabriel Egídio Santos Beloni

Gabriel Evangelista Massara

Pedro Henrique Cardoso Maia

Thiago Aurelio Nunes Martins

Sumário

C++

1. HISTÓRICO		
01	Introdução - <i>origem, filosofia e contexto</i>	03
02	Linha do Tempo — <i>1979 a C++23</i>	04
03	Genealogia e Linguagens Relacionadas	05
2. PARADIGMAS E ORIENTAÇÃO A OBJETOS		
04	Múltiplos Paradigmas — <i>procedural, genérico, OOP, funcional</i>	07
05	Abstração e Encapsulamento	08
06	Herança — <i>simples, múltipla e virtual</i>	10
07	Polimorfismo — <i>virtual, override, vtable</i>	11
3. CARACTERÍSTICAS DA LINGUAGEM		
08	Gerenciamento Manual de Memória	12
09	Templates e Programação Genérica	13
10	Alto Desempenho e C++ em jogos — <i>zero-cost abstractions</i>	14
4. PRÁTICA E CONSIDERAÇÕES		
11	História das ferramentas e Prática	16
12	Considerações Finais	18
13	Apêndice e Referências	19

I Introdução

- C++ é uma linguagem de programação de propósito geral, criada por **Bjarne Stroustrup** em 1979 nos laboratórios Bell.
- Originalmente chamada “C with Classes”, foi renomeada para C++ em 1983.
- Estende a linguagem C adicionando recursos de **orientação a objetos, programação genérica e metaprogramação**.
- Amplamente utilizada em sistemas operacionais, motores gráficos, jogos, banco de dados, sistemas embarcados e software de alto desempenho.
- Padronizada internacionalmente pela ISO/IEC desde 1998, com atualizações regulares a cada 3 anos.
- Filosofia: “Você não paga por aquilo que não usa” (zero-overhead principle).

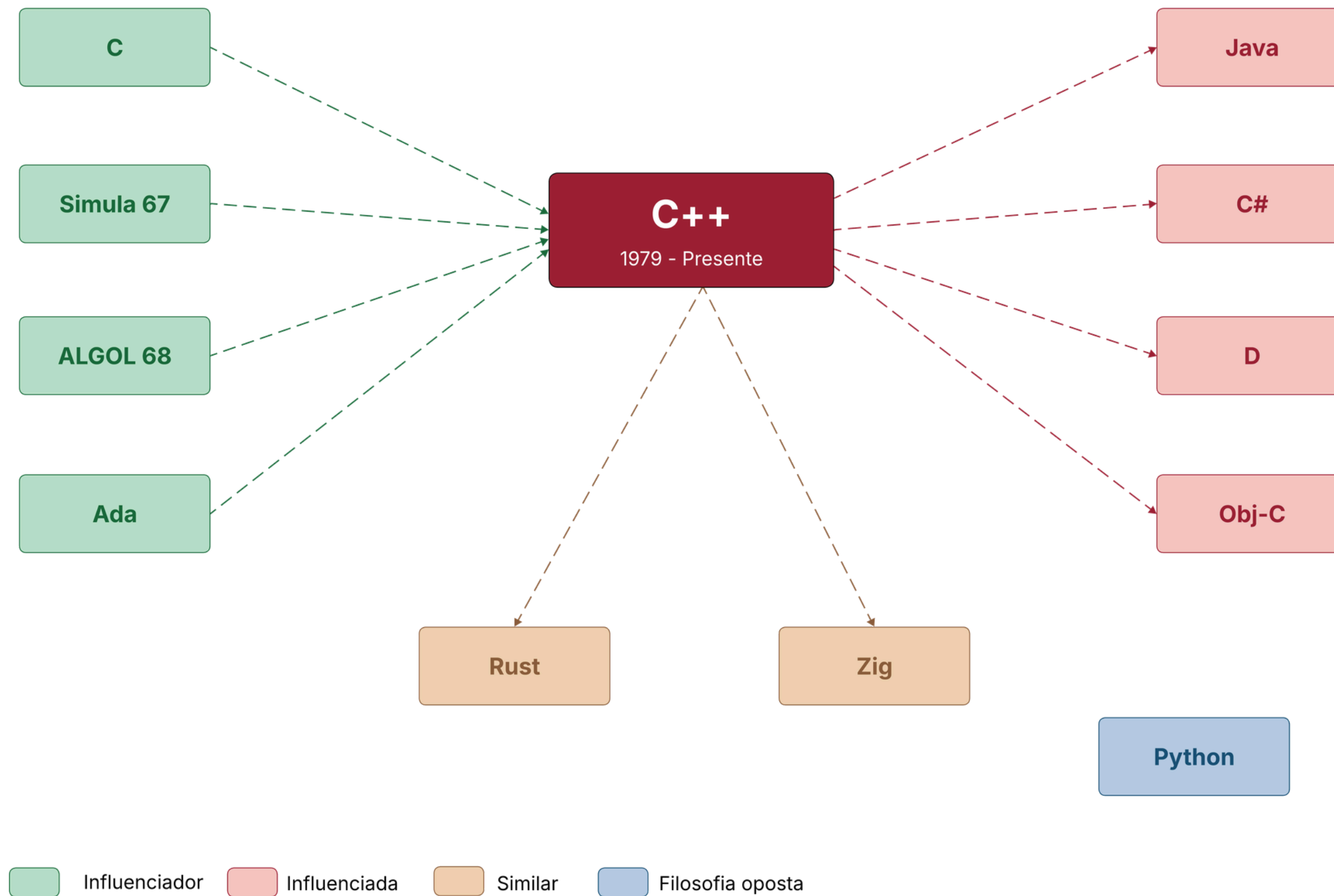
| Linha do Tempo C++

- 1979**
Bjarne Stroustrup inicia o "C with Classes"
- 1983**
Renomeado para C++
- 1985**
Primeira edição comercial (The C++ Programming Language)
- 1989**
Adicionadas classes abstratas, herança múltipla, templates (2.0) e tratamento de exceções (3.0) à linguagem.
- 1998**
Padronização ISO/IEC 14882:1998 (C++98)

- 2011**
Revolução da linguagem: auto, lambdas, move semantics, smart pointers, threads, range-for, nullptr, initializer lists, constexpr, regex.
- 2020 - Major**
C++20 — Módulos, corrotinas, ranges, concepts
- 2023 - Atual**
C++23 — Aperfeiçoamentos e novas funcionalidades

Genealogia

Mapa visual da herança e derivações do C++ no ecossistema de linguagens.



| Linguagens Relacionadas

Influências, herdeiros e contemporâneos do C++ no panorama das linguagens de programação.

▲ INFLUENCIADORES

Linguagens que fundamentaram a criação do C++, fornecendo sintaxe, paradigmas ou mecanismos que foram adotados e estendidos.

- C - base sintática e de memória
- ALGOL 68 - estrutura de controle
- ML - Templates e tipos genéricos
- Simula 67 - classes e heranças
- Ada - sobrecarga de métodos

▼ INFLUENCIADAS

Linguagens que nasceram sob forte influência do C++ ou foram projetadas como alternativas com sintaxe e conceitos derivados.

- Java - POO, sintaxe, collections
- C# - POO + managed runtime
- Objective-C - POO em torno de C
- PHP - sintaxe de expressões

▬ SIMILARES / CONCORRENTES

Linguagens de nicho similar — sistemas de baixo nível, performance crítica — que competem ou complementam o C++.

- Rust - Segurança de memória sem GC
- Zig - Alternativa minimalista do C
- Fortran - HPC, computação numérica

Os Múltiplos Paradigmas do C++



Procedural

Estrutura direta herdada do C. Foco no controle sequencial por funções, garantindo total controle de memória física e mapeamento limpo de registradores de hardware

Funções

Ponteiros

Stack/Heap



Genérico

Abstração de tipos através de Templates. Permite criar algoritmos globais (como a Standard Template Library - STL) resolvidos direto em tempo de compilação.

Templates

STL

Concepts



Orientado a Objetos

O foco central da expansão. Permite modelar o software mapeando de forma exata entidades reais em objetos seguros, reutilizáveis e fáceis de arquitetar.

Classes

Polimorfismo

Herança



Funcional

Fortalecido no C++11 com expressões lambda e funções de alta ordem. Permite composição de operações sem efeitos colaterais sobre estados externos.

Lambdas

std::function

Ranges

| Orientação a Objetos em C++

Pilar 1: Abstração

Esconder Detalhes Complexos

Abstração foca em definir o "o que" um objeto faz, ocultando a lógica do "como" faz.

Em C++, isso é obtido de forma estrita usando Classes Abstratas contendo funções virtuais puras.

- ✓ Impede que instâncias diretas da classe base sejam criadas.
- ✓ Define contratos que as classes herdeiras são obrigadas a respeitar.

```
// Abstração: expõe APENAS a interface pública.  
// Detalhes internos ficam completamente ocultos.  
  
class ContaBancaria {  
public:  
    // Interface pública – o que o usuário precisa saber  
    void depositar(double valor);  
    void sacar(double valor);  
    double getSaldo() const;  
  
private:  
    // Implementação oculta – o usuário não acessa  
    diretamente  
    double saldo = 0.0;  
    std::vector<std::string> historico;  
  
    void registrarTransacao(const std::string& tipo,  
double v);  
};  
  
int main() {  
    ContaBancaria conta;  
    conta.depositar(1000.0); // uso simples  
    conta.sacar(250.0);      // complexidade oculta  
    std::cout << conta.getSaldo(); // imprime: 750  
}
```


| Orientação a Objetos em C++

Pilar 2: Encapsulamento

Blindagem e Modificadores de Acesso

Agrupa atributos e rotinas dentro de um objeto, controlando o nível de acesso direto do cliente.

O estado interno deve permanecer oculto para evitar consistências inválidas de dados.

 **private:** Dados expostos exclusivamente para a classe proprietária.

 **public:** Métodos seguros de acesso (Getters/Setters).

```
// Encapsulamento: dado protegido,  
// acesso controlado via getters/setters.  
  
class Temperatura {  
public:  
    void setCelsius(double c) {  
        if (c < -273.15)  
            throw std::invalid_argument("Abaixo do zero absoluto!");  
        celsius = c;  
    }  
  
    double getCelsius() const { return celsius; }  
    double getFahrenheit() const { return celsius * 9.0 / 5.0 + 32; }  
    double getKelvin() const { return celsius + 273.15; }  
  
private:  
    double celsius = 0.0; // dado protegido – não acessível  
    diretamente  
};  
  
int main() {  
    Temperatura t;  
    t.setCelsius(100.0);  
    std::cout << t.getFahrenheit() << "°F\n"; // 212°F  
    std::cout << t.getKelvin() << "K\n"; // 373.15K  
    // t.celsius = -999; ← ERRO de compilação: membro privado  
}
```

| Orientação a Objetos em C++

Pilar 3: Herança

Reutilização e Extensibilidade

Permite que uma classe (derivada) herde estruturas e dados de outra classe de nível superior (base).

Exclusividade C++: Suporta de forma nativa a Herança

Múltipla, onde uma classe filha pode herdar comportamento de duas bases independentes.

↑ Herança pública (:public) espelha relações clássica “1:N”

↑ Evita duplicações em larga escala do código-fonte.

```
// Herança: classe filha reutiliza e estende a classe pai.

class Veiculo {
protected:
    std::string marca;
    int ano;
public:
    Veiculo(const std::string& m, int a) : marca(m), ano(a) {}
    void exibirInfo() const {
        std::cout << marca << " (" << ano << ")\n";
    }
};

// Herança simples
class Carro : public Veiculo {
    int numPortas;
public:
    Carro(const std::string& m, int a, int p)
        : Veiculo(m, a), numPortas(p) {}
    void exibirInfo() const {
        Veiculo::exibirInfo();
        std::cout << "Portas: " << numPortas << "\n";
    }
};

// Herança múltipla
class Barco : public Veiculo { /* ... */ };
class Anfibio : public Carro, public Barco { /* ... */ };
```

| Orientação a Objetos em C++

Pilar 4: Polimorfismo

Multicomportamento Dinâmico

Capacidade de objetos de diferentes classes derivadas responderem à mesma assinatura de método de maneiras distintas.

Em C++, é implementado usando ponteiros de classe base aliados às palavras-chave virtual e override.

 Ligação Dinâmica (Runtime Binding).

 Crucial para construir arquiteturas escaláveis e desacopladas

```
// Polimorfismo: mesma interface, comportamentos diferentes em runtime.

class Forma {
public:
    virtual double area() const = 0; // método virtual puro
    virtual std::string nome() const = 0;
    virtual ~Forma() = default;
};

class Circulo : public Forma {
    double r;
public:
    Circulo(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    std::string nome() const override { return "Circulo"; }
};

class Retangulo : public Forma {
    double w, h;
public:
    Retangulo(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
    std::string nome() const override { return "Retangulo"; }
};

int main() {
    std::vector<Forma*> formas = { new Circulo(5.0), new
    Retangulo(4.0, 6.0) };

    for (auto f : formas)
        std::cout << f->nome() << ": " << f->area() << " u²\n";
    // Circulo: 78.5397 u²
    // Retangulo: 24 u²
}
```

CARACTERÍSTICA 01

Gerenciamento Manual de Memória

Ao contrário de linguagens gerenciadas (Java, Python), C++ oferece controle direto sobre alocação e desalocação de memória. Isso permite máximo desempenho, porém oferece risco. O C++ 11+ introduziu smart pointers para mitigar erros comuns.

```
// — Alocação manual (legado)

int* ptr = new int(42);
delete ptr; // obrigatório! sem isso = memory leak

// — Smart pointer: unique_ptr — C++11

// Propriedade exclusiva. Deletado automaticamente
// ao sair do escopo — sem delete manual.
std::unique_ptr<int> sptr = std::make_unique<int>(42);
```

Templates & Genéricos

Templates permitem escrever código parametrizado por tipos, avaliado em tempo de compilação. Permitem desde funções genéricas simples até metaprogramação avançada, sem overhead de runtime.

- Function Templates— funções que funcionam com qualquer tipo
- Class Templates— estruturas de dados genéricas (ex: `std::vector<T>`)
- Concepts— restrições para templates com mensagens de erro legíveis (C++20)
- Template Metaprogramming (TMP) — computação em tempo de compilação

```
// — Template de função
template<typename T>
T minimo(T a, T b) { return a < b ? a : b; }

// — Template de classe

std::vector<int> numerosInteiros = {10, 20, 30};
std::vector<float> numerosDecimais= {10.00, 20.31, 30.14};

// — Concept (C++20) - restrição explícita de tipo

template<typename T>
concept Numerico = std::is_arithmetic_v<T>;

template<Numerico T>
T quadrado(T x) { return x * x; }

// — Uso
// T deduzido = int
minimo(3, 2);
minimo(3.14, 2.71); // T deduzido = double
quadrado(5);        // 25
```


CARACTERÍSTICA 03

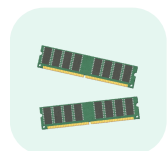
Alto Desempenho

C++ compila diretamente para código de máquina nativo, sem overhead de runtime como GC ou JIT. É a escolha padrão para sistemas embarcados, engines de jogos, HPC e sistemas operacionais.



Zero cost abstractions

Templates e inline expandidos em compile-time sem overhead de chamadas virtuais desnecessárias.



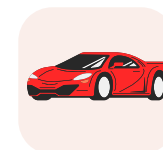
Controle de Hardware

Acesso direto à memória, SIMD intrinsics, ponteiros e aritmética de baixo nível.



Sem GC

Sem pausas de garbage collection — previsibilidade de latência crítica para sistemas de tempo real.



Otimizações do compilador

GCC/Clang/MSVC aplicam inlining, loop unrolling, auto-vetorização e LTO automaticamente.

CARACTERÍSTICA 04

C++ e jogos modernos

POR QUE C++ DOMINA

Performance de missão crítica

Jogos processam física, gráficos, IA e rede simultaneamente a cada frame. C++ entrega controle direto sobre memória e hardware sem overhead de runtime.



Unreal Engine

100% C++ — usado em Fortnite, The Witcher, Valorant



Counter-Strike 2

Motor Source 2 em C++. Sub-tick system requer precisão de microssegundos no registro de dano e movimento — impossível com GC.



Godot

É escrito em C++ ultra-otimizado, opensource.



Valorant

Unreal Engine 4 (C++). Anti-cheat Vanguard e sistema de habilidades com lógica de colisão em tempo real exigem latência determinística.

HISTÓRIA DAS FERRAMENTAS

IDEs e editores ao longo da evolução do C++

O ambiente de desenvolvimento acompanhou cada era do C++. De terminais sem cor aos editores modernos com LSP, cada ferramenta refletia as possibilidades do hardware da época.

ERA DOS TERMINAIS · 1979 – 1995

Terminal

vi / Emacs

1976 – 1991

Editores de texto puro. O próprio Bjarne Stroustrup usava Emacs com scripts customizados para editar os primeiros fontes do C++.

IDE

Turbo C++

1990 – 1997

Borland. Primeira IDE popular com compilador integrado. Amplamente usada no ensino. Interface TUI azul icônica no MS-DOS.

IDE

Borland C++

1991 – 1998

Sucessora do Turbo C++. Suportava Windows 3.x. Pioneira na integração de debugger visual com código-fonte.

ERA DAS IDES GRÁFICAS · 1995 – 2010

IDE

Visual C++ 6.0

1998

Microsoft. Padrão corporativo por quase uma década. Autocomplete básico (IntelliSense 1.0) e integração com COM/ActiveX.

IDE

Code::Blocks

2005

Open source, multiplataforma. Integração nativa com GCC e MinGW. Substituiu o Turbo C++ no ambiente acadêmico.

IDE

Eclipse CDT

2007

Plugin C/C++ para Eclipse. Usado extensamente em projetos embedded e corporativos com GCC cruzado.

ERA MODERNA · 2010 – PRESENTE

IDE

CLion

2015 – hoje

JetBrains. IDE dedicada a C/C++ com análise estática de código, refactoring inteligente e integração com CMake e GCC/Clang.

Editor

VS CODE + clangd

2016 – hoje

Editor utilizado nesta demonstração. Extensão C/C++ da Microsoft + servidor de linguagem clangd para autocomplete e análise em tempo real.

IDE

Visual Studio 2022

2021 – hoje

Microsoft. Padrão em desenvolvimento Windows e games com Unreal Engine. IntelliSense avançado e profiler integrado.

Setup utilizado hoje

EDITOR PRINCIPAL

Visual Studio Code

Editor leve e extensível da Microsoft. Suporte a C++ via extensão oficial com IntelliSense, debugging com GDB, integração com terminal e controle de versão.

C/C++ Extension

clangd LSP

GDB Debugger

CMake Tools



COMPILADOR

GCC 13.2 · g++

GNU Compiler Collection. Padrão acadêmico e de produção em ambientes Linux. Suporte completo a C++17 e parcial a C++23.

C++17

-O2

-Wall

SISTEMA OPERACIONAL

Linux / Ubuntu

Ambiente nativo do GCC. Terminal integrado com compilação direta, sem camadas de abstração entre o código e o binário gerado.

Exemplo: Fatorial (Comparações)

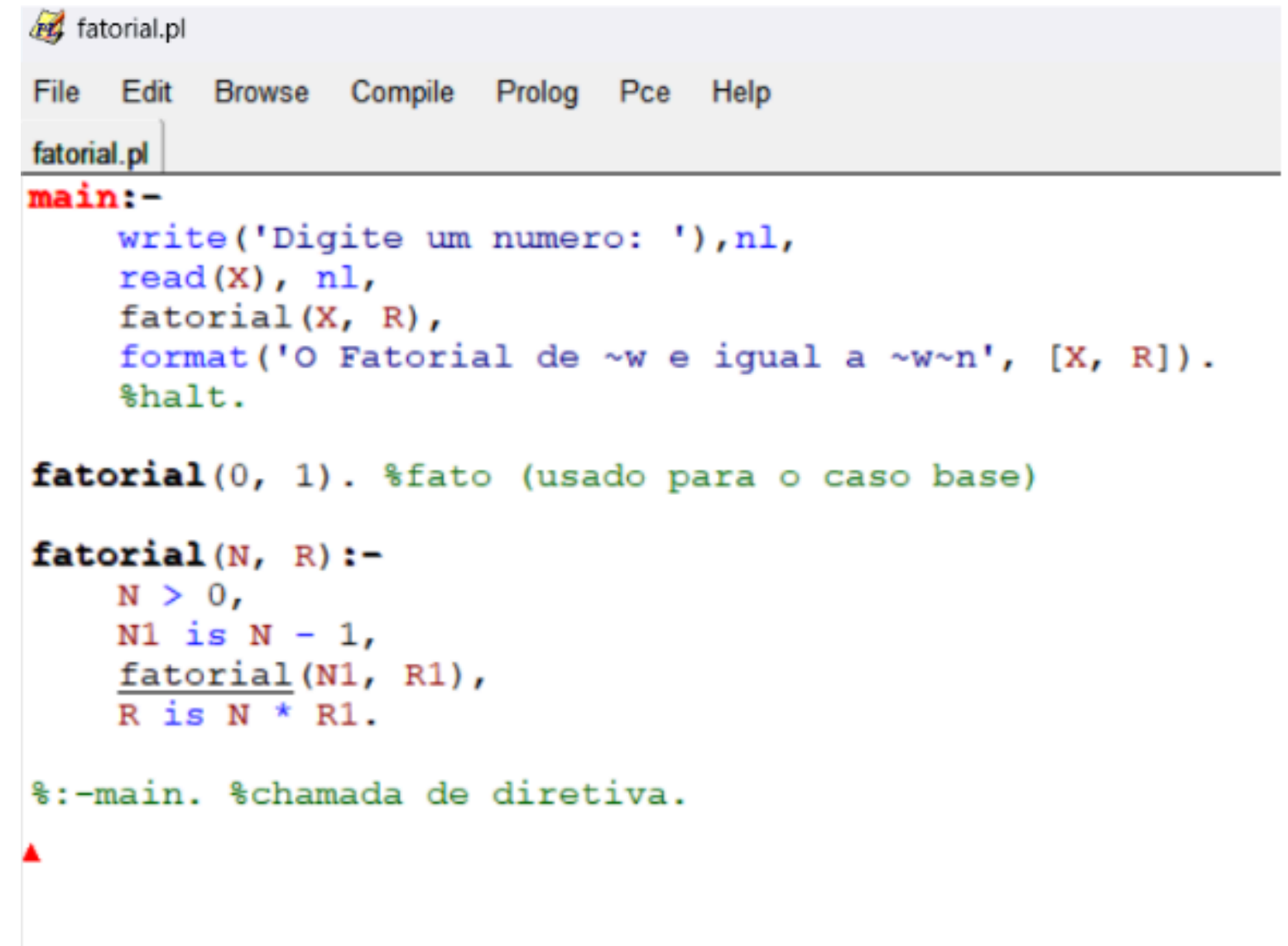
C++

```
#include <iostream>

constexpr unsigned long long fatorial(int n) {
    return (n <= 1) ? 1 : n * fatorial(n - 1);
}

int main() {
    int num;
    std::cout << "Digite um numero: ";
    if (std::cin >> num && num >= 0) {
        std::cout << "O Fatorial de " << num << " e igual a " << fatorial(num) << "\n";
    } else {
        std::cerr << "Entrada invalida.\n";
    }
    return 0;
}
```

Prolog



```
fatorial.pl
File Edit Browse Compile Prolog Pce Help
fatorial.pl
main:-
    write('Digite um numero: '),nl,
    read(X), nl,
    fatorial(X, R),
    format('O Fatorial de ~w e igual a ~w~n', [X, R]).
    %halt.

fatorial(0, 1). %fato (usado para o caso base)

fatorial(N, R):-
    N > 0,
    N1 is N - 1,
    fatorial(N1, R1),
    R is N * R1.

%:-main. %chamada de diretiva.
```

| Considerações Finais

C++ é onipresente

- C++ permanece como uma das linguagens mais poderosas e versáteis da computação
- Utilizada em sistemas críticos: NASA, sistemas financeiros, motores de jogos (Unreal Engine), navegadores (Chrome V8).
- A evolução constante (C++20, C++23, futura C++26) mantém a linguagem moderna e competitiva.
- Oferece controle absoluto sobre recursos computacionais — ideal para software de alto desempenho.
- Comunidade global ativa e vasta oferta de bibliotecas (Boost, Qt, OpenCV).
- Aprender C++ proporciona uma base sólida para entender profundamente como computadores funcionam.

"Existem apenas dois tipos de linguagens: aquelas das quais as pessoas reclamam e aquelas que ninguém usa." — Bjarne Stroustrup

| Apêndice

Gabriel Egídio Santos Beloni

Gosto bastante de C++ por ser uma linguagem rápida, direta e muito próxima do C, o que permite ter mais controle sobre o funcionamento do programa e entender melhor como o computador executa cada processo. Além disso, ela oferece alto desempenho e é muito utilizada em áreas que exigem eficiência, como jogos, sistemas e softwares de baixo nível. Considero uma ótima linguagem para desenvolver lógica de programação e adquirir uma base técnica sólida como programador.

| Apêndice

Gabriel Evangelista Massara

Durante este trabalho, foi possível perceber que o C++ continua sendo uma linguagem muito relevante, mesmo tendo sido criada há décadas. Ele se destaca por aproximar o programador do funcionamento real do computador, principalmente no controle de memória, desempenho e uso eficiente do hardware.

O estudo também mostrou que o C++ é uma linguagem multiparadigma. Ela permite programar de forma procedural, orientada a objetos, genérica e funcional. Na orientação a objetos, os conceitos de abstração, encapsulamento, herança e polimorfismo ajudam a organizar programas maiores e tornam o código mais reutilizável.

Por fim, entendo que aprender C++ fortalece a base de qualquer programador, porque exige atenção, lógica e responsabilidade. Além disso, sua presença em jogos, sistemas embarcados, motores gráficos e aplicações de alto desempenho mostra que a linguagem ainda tem grande importância no mercado atual.

| Apêndice

Pedro Henrique Cardoso Maia

Embora a orientação a objetos não seja meu paradigma favorito, tenho grande afinidade com o C++ devido à sua velocidade. Gosto do desenvolvimento de baixo nível, de trabalhar direto na manipulação de bits e extrair o máximo de desempenho do programa. Além disso, algo que me chamou muita atenção na linguagem é o poder da metaprogramação proporcionada pelos templates. Recomendo fortemente o C++: tanto pela evolução que ele proporciona na bagagem como programador, quanto pela sua relevância e presença no mercado atual.

| Apêndice

Thiago Aurelio Nunes Martins

A pesquisa sobre o C++ me fez gostar bastante da linguagem pois a programação orientada a objetos permite aplicar os conceitos de Abstração, Encapsulamento, Herança e Polimorfismo de forma direta e segura e conseguir estruturar programas complexos com essa organização da POO, sem perder o controle da memória e o alto desempenho. É uma linguagem que ao aprender ensina uma base sólida de como o computador funciona e possui uma enorme relevância no mercado.

| Referências Bibliográficas

- ISO/IEC. ISO/IEC 14882:2020 Programming Languages — C++. Geneva: International Organization for Standardization, 2020.
- STROUSTRUP, Bjarne. A Linguagem de Programação C++. 4. ed. Porto Alegre: Bookman, 2015.
- CPLUSPLUS. The C++ Resources Network. Disponível em: Referências Bibliográficas. Acesso em: 23 maio 2026.
- CPPREFERENCE. C++ reference. Disponível em: Referências Bibliográficas. Acesso em: 23 maio 2026.

Dúvidas & Perguntas

```
std::cout << “Muito Obrigado!” << std::endl;
```